

Agenda
Strings
Lists
Dicts
Tuples
Sets
Functions

String

- Sequence of Characters
- "'", '"', '\', '\n', '\t', '\r', '\f', '\o'
- immutable
- Indexing, slicing
- There are methods associated with string

```
>>> x = "VISHWAS K SINGH"
>>> len(x)
15
>>> x[0]
'V'
>>> x[1]
'I'
>>> x[2]
'S'
>>> x[13]
'G'
>>>
```

```
>>> x[-1]
'H'
>>> x[-2]
'G'
>>> x[-3]
'N'
>>>
```

```
>>> x1 = "ABBA"
>>> x2 = "ABBA"
>>> x1[0] == x2[-1]
True
>>> x1[1] == x2[-2]
True
>>>
```

```
>>> x = "VISHWAS K SINGH"
>>> x[0:4]
'VISH'
>>> #WAS
>>> x[4:6]
'WA'
>>> x[4:7]
'WAS'
>>> #SING
>>> x[10:14]
'SING'
>>>
```

For Indexing, Always from left to right, except when step is -ve

```
>>> x = "VISHWAS K SINGH"
>>> x[-15:-11]
'VISH'
>>> #WAS
>>> x[-11:-8]
'WAS'
>>> #SING
>>> x[-5:-1]
'SING'
>>> x[-5:0]
''
```

```
>>> x = "VISHWAS K SINGH"
>>> x[::1]
'VISHWAS K SINGH'
>>> x[::2]
'VSWSKSNH'
>>> x[::3]
'VHS N'
>>> x[::4]
'VWKN'
```

```
>>> x = "VISHWAS K SINGH"
>>> x[0:-11]
'VISH'
>>> x[-15:4]
'VISH'
>>> #WAS
>>>
```

```
>>> x[::-1]
'HGNIS K SAWHSIV'
>>> x[::-2]
'HNSKSWSV'
>>> x[::-3]
'HIKAS'
>>> x1
'ABBA'
>>> rev_x1 = x1[::-1]
>>> x1 == rev_x1
True
>>>
```

```
>>> x = "VISHWAS K SINGH"
>>> x[0:4]
'VISH'
>>> x[:4]
'VISH'
>>> x[10:]
'SINGH'
>>> len(x)
15
>>> x[14]
'H'
>>> x[15]
Traceback (most recent call last):
  File "<python-input-38>", line 1, in <module>
    x[15]
    ~^^^
IndexError: string index out of range
>>> x[10:19]
'SINGH'
>>> x[10:900]
'SINGH'
>>> |
```

```
>>> x = "VISHWAS K SINGH"
>>> x[6:3:-1]
'SAW'
>>> x[-9:-12:-1]
'SAW'
>>> x[6:-12:-1]
'SAW'
>>> x[-9:3:-1]
'SAW'
>>> |
```

```
>>> x = "VISHWAS K SINGH"
>>> x[2:4]
'SH'
>>> x[-14:-15:-1]
'I'
>>> x[-14:-16:-1]
'IV'
>>> new_name = x[2:4] + x[-14:-16:-1]
>>> new_name
'SHIV'
>>>
```

String Methods

```
>>> dir(str)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getat
tribute__', '__getitem__', '__getnewargs__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__i
ter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr
__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'cent
er', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isasci
i', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'joi
n', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex',
'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate
', 'upper', 'zfill']
```

```
>>> x = "VISHWAS K SINGH"
>>> x.lower()
'vishwas k singh'
>>> x.capitalize()
'Vishwas k singh'
>>> x.title()
'Vishwas K Singh'
>>> x.rjust(80, '-')
'-----VISHWAS K SINGH'
>>> x.ljust(80, '-')
'VISHWAS K SINGH-----'
>>> x.center(80, '-')
'-----VISHWAS K SINGH-----'
>>> x.find('I')
1
>>> x.find('I', 2)
11
>>> x.count('I')
2
>>> x.count('S')
3
>>> x.split()
['VISHWAS', 'K', 'SINGH']
>>> list(x)
['V', 'I', 'S', 'H', 'W', 'A', 'S', ' ', 'K', ' ', 'S', 'I', 'N', 'G', 'H']
>>> x1 = list(x)
>>> ''.join(x1)
'VISHWAS K SINGH'
>>>
```

```
>>> email = "vishwas@cloudthat.com"
>>> email.find('@')
7
>>> email.count('@') == 1
True
>>> email[:email.find('@')]
'vishwas'
>>>
```

Indexes and Positions for slicing

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
V	I	S	H	W	A	S		K		S	I	N	G	H
-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
VISH	0,4		WAS	4,7		SING	10,14		SAW	6,3				
	-15,-11			-11,-8			-5,-1			-9,-12				
	0,-11			4,-8			10,-1			6,-12				
	-15,4			-11,7			-4,14			-9,3				

Debugging in Python

1. Print Statement Debugging

```
07_uname_extractor.py > ...  
1  email = input("Enter your email ID: ")  
2  
3  print(email.find("@"))  
4  if not email.find('@') != -1 and email.count('@') == 1:  
5      print(email.find("@"))  
6      print("Invalid Email ID")  
7  else:  
8      print("Valid Email ID")
```

Naive way

2. Python Debugger – pdb

07_uname_extractor.py > ...

```
1 email = input("Enter your email ID: ")
2
3 print(email.find("@"))
4 if not email.find('@') != -1 and email.count('@') == 1:
5     print(email.find("@"))
6     print("Invalid Email ID")
7 else:
8     print("Valid Email ID")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRESQL QUERY RESULTS

```
'vishwascom'
(Pdb) p email.find('@')
-1
(Pdb) n
-1
> c:\users\vishwas Singh\workspace\ey-coh8-workspace\07_uname_extractor.py(4)<module>()
-> if not email.find('@') != -1 and email.count('@') == 1:
(Pdb) n
> c:\users\vishwas Singh\workspace\ey-coh8-workspace\07_uname_extractor.py(8)<module>()
-> print("Valid Email ID")
(Pdb) p not email.find('@')
False
(Pdb) c
Valid Email ID
The program finished and will be restarted
> c:\users\vishwas Singh\workspace\ey-coh8-workspace\07_uname_extractor.py(1)<module>()
-> email = input("Enter your email ID: ")
(Pdb) q
```

3. VSCODE Debugging

The screenshot displays the Visual Studio Code (VS Code) interface during a Python debugging session. The workspace is named 'ey-coh8-workspace'.

Left Sidebar (Debug Console):

- VARIABLES:**
 - Locals:**
 - special variables: email = 'vishwascom'
 - Globals:**
 - special variables: email = 'vishwascom'
- WATCH:**
 - email.find('@') != -1 = False
 - email.count('@') == 1 = False
 - email.find('@') = -1
- CALL STACK:** Running
- BREAKPOINTS:**
 - ☐ Raised Exceptions
 - ☒ Uncaught Exceptions
 - ☐ User Uncaught Exceptions
 - ☒ 07_uname_extractor.py (3)
 - ☒ 07_uname_extractor.py (6)

Main Editor:

The file '07_uname_extractor.py' is open, showing the following code:

```
1 email = input("Enter your email ID: ")
2
3 if not email.find('@') != -1 and email.count('@') == 1:
4     print("Invalid Email ID")
5 else:
6     print("Valid Email ID")
```

The code is currently paused at line 3, which is highlighted in green. A red box highlights the debug toolbar above the code, showing icons for stepping through the code (step over, step into, step out, step back, step forward) and a red square icon.

Terminal:

The terminal shows the command to run the script and the input provided:

```
Microsoft Windows [Version 10.0.26200.9106]
(c) Microsoft Corporation. All rights reserved.

C:\Users\VishwasKSingh\Workspace\ey-coh8-workspace> cmd /C "c:\Apps\Python3\python.exe c:\Users\VishwasKSingh\
.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher 57693 -- C:\Users\Vishwa
sKSingh\Workspace\ey-coh8-workspace\07_uname_extractor.py "
Enter your email ID: vishwascom

```

Lab: IPv4 Address Validator

1. Dotted Decimal Notation has 4 fields exactly
2. Must contain only numbers between 0 and 255

Eg:

ipv4: 192.168.1.1 → Valid

ipv4: a.b.1.2 → Invalid

ipv4: 192.294.1.1 → Invalid

ipv4: 192.1.1 → Invalid

Lists

```
>>> l1 = []
>>> l2 = list()
>>> type(l2)
<class 'list'>
>>> l3 = ["apple", "mango", 12, 4.5, {1: "apple"}, [23, 45], (1, 2, 3)]
>>> l3
['apple', 'mango', 12, 4.5, {1: 'apple'}, [23, 45], (1, 2, 3)]
>>> l2.append(23)
>>> l2
[23]
>>> l2.append(23)
>>> l2.append(23)
>>> l2
[23, 23, 23]
>>> l2.insert(1, 89)
>>> l2
[23, 89, 23, 23]
>>> l2.pop()
23
>>> l2
[23, 89, 23]
>>> l2.remove(89)
>>> l2
[23, 23]
>>> len(l2)
2
```

```
>>> for i in range(len(l3)):
...     print(i, l3[i])
...
0 apple
1 mango
2 12
3 4.5
4 {1: 'apple'}
5 [23, 45]
6 (1, 2, 3)
>>> for item in l3:
...     print(item)
...
apple
mango
12
4.5
{1: 'apple'}
[23, 45]
(1, 2, 3)
>>> |
```

List Methods

```
>>> l3
['apple', 'mango', 12, 4.5, {1: 'apple'}, [23, 45], (1, 2, 3)]
>>> l2
[23, 23]
>>> l4 = l2
>>> l5 = l2.copy()
>>> i(l2)
Traceback (most recent call last):
  File "<python-input-109>", line 1, in <module>
    i(l2)
    ~^^^
TypeError: 'int' object is not callable
>>> id(l2)
2150154408320
>>> id(l4)
2150154408320
>>> id(l5)
2150154411968
>>> l2.append(90)
>>> l2
[23, 23, 90]
>>> l4
[23, 23, 90]
>>> l5
[23, 23]
```

```
>>> l3.remove("apple")
>>> l3
['mango', 12, 4.5, {1: 'apple'}, [23, 45], (1, 2, 3)]
>>> l2.index(23)
0
>>> l2.find(23)
Traceback (most recent call last):
  File "<python-input-120>", line 1, in <module>
    l2.find(23)
    ~~~~~~
AttributeError: 'list' object has no attribute 'find'
>>> l2.count(23)
2
>>> l6 = []
>>> for i in range(50):
...     if i%2==0:
...         l6.append(i)
...
>>> l6
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48]
>>> l6.reverse()
>>> l6
[48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16, 14, 12, 10, 8, 6, 4, 2, 0]
>>> l6.sort()
>>> l6
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48]
>>>
```

```
>>> l6[0]
0
>>> l6[1]
2
>>> l6[2]
4
>>> l6[-1]
48
>>> l6[-2]
46
>>> l6[-3]
44
>>> l6[:4]
[0, 2, 4, 6]
>>> l6[8:]
[16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48]
>>> l6[::-1]
[48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16, 14, 12, 10, 8, 6, 4, 2, 0]
>>> l6[::-2]
[48, 44, 40, 36, 32, 28, 24, 20, 16, 12, 8, 4, 0]
>>> l6[::-3]
[48, 42, 36, 30, 24, 18, 12, 6, 0]
>>> l6[:3]
[0, 6, 12, 18, 24, 30, 36, 42, 48]
```

List Methods

```
>>> l2
[23, 23, 90]
>>> l2.extend(l6)
>>> l2
[23, 23, 90, 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48]
>>> l2.clear()
>>> l2
[]
>>> l3
['mango', 12, 4.5, {1: 'apple'}, [23, 45], (1, 2, 3)]
>>> del l3
>>> l3
Traceback (most recent call last):
  File "<python-input-150>", line 1, in <module>
    l3
NameError: name 'l3' is not defined. Did you mean: 'l1'?
>>> l2
[]
>>> l6
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48]
>>> del l6[8]
>>> len(l6)
24
>>> l3 = l6 + l5
>>> l3
[0, 2, 4, 6, 8, 10, 12, 14, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 23, 23]
>>> l5
[23, 23]
>>>
```

Scenario:

You have 10 Numbers which represent Number of failed Login counts. And another list that contains ip_addresses. Find out What is the 2nd highest count of failed logins

failed_counts = [23,45,67,98,2,75,100]

```
>>> failed_counts = [23,45,67,98,2,75,100]
```

```
>>> failed_counts.sort()
```

```
>>> failed_counts[-2]
```

```
98
```

```
>>> failed_counts
```

```
[2, 23, 45, 67, 75, 98, 100]
```

```
>>>
```

dictionary

- key value pairs
- Hash maps
- Keys are unique
- Recommended to have strings and integers as keys

```
>>> d1 = {}
>>> type(d1)
<class 'dict'>
>>> d2 = dict()
>>> type(d2)
<class 'dict'>
>>> d3 = {"a": "Vishwas", "b": "arjun", "inventory": ["laptop", "Keyboard"]}
>>> d3
{'a': 'Vishwas', 'b': 'arjun', 'inventory': ['laptop', 'Keyboard']}
>>> d3["a"]
'Vishwas'
>>> d3["b"]
'arjun'
>>> d3["c"]
Traceback (most recent call last):
  File "<python-input-171>", line 1, in <module>
    d3["c"]
    ~~~~~^
KeyError: 'c'
>>> dir(dict)
['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__form
at__', '__ge__', '__getattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__
', '__ior__', '__iter__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__reduce__', '__reduce_ex__', '__
repr__', '__reversed__', '__ror__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'clear'
, 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
```

Dictionary Manipulation

```
>>> d3
{'a': 'Vishwas', 'b': 'arjun', 'inventory': ['laptop', 'Keyboard']}
>>> d3["c"]
Traceback (most recent call last):
  File "<python-input-183>", line 1, in <module>
    d3["c"]
    ~~^~^~^
KeyError: 'c'
>>> d3.get("b", "Not Available")
'arjun'
>>> d3.get("c", "Not Available")
'Not Available'
>>> d3["c"] = "Riya"
>>> d3
{'a': 'Vishwas', 'b': 'arjun', 'inventory': ['laptop', 'Keyboard'], 'c': 'Riya'}
>>> d3.keys()
dict_keys(['a', 'b', 'inventory', 'c'])
>>> d3.values()
dict_values(['Vishwas', 'arjun', ['laptop', 'Keyboard'], 'Riya'])
>>> d3.items()
dict_items([('a', 'Vishwas'), ('b', 'arjun'), ('inventory', ['laptop', 'Keyboard']), ('c', 'Riya')])
>>> for k in d3.keys():
...     print(k, d3[k])
...
a Vishwas
b arjun
inventory ['laptop', 'Keyboard']
c Riya
>>> |
```

Dictionary Methods

```
>>> for v in d3.values():
...     print("Value: ", v)
...
Value: Vishwas
Value: arjun
Value: ['laptop', 'Keyboard']
Value: Riya
>>> for k,v in d3.items():
...     print(k,'|', v)
...
a | Vishwas
b | arjun
inventory | ['laptop', 'Keyboard']
c | Riya
>>> |
```

```
>>> d3
{'a': 'Vishwas', 'b': 'arjun', 'inventory': ['laptop', 'Keyboard'], 'c': 'Riya'}
>>> d3.pop("c")
'Riya'
>>> d3
{'a': 'Vishwas', 'b': 'arjun', 'inventory': ['laptop', 'Keyboard']}
>>> d3.popitem()
('inventory', ['laptop', 'Keyboard'])
>>> d3
{'a': 'Vishwas', 'b': 'arjun'}
```

```
>>> l2 = [('Mouse', 20), ('Keyboard', 5), ('laptop', 10), ('monitor', 100)]
>>> inventory
{'Mouse': 0, 'keyboard': 0, 'laptop': 0}
>>> inventory.update(l2)
>>> inventory
{'Mouse': 20, 'keyboard': 0, 'laptop': 10, 'Keyboard': 5, 'monitor': 100}
>>>
```

Lab: Take a string from the user, count the frequency of characters in the string

Eg: "hello"

{"h":1,"e":1,"o":1,"l":2}

Lab: You have a Log of Login Attempts and Ip addresses, success / Failure
Calculate Number of Login Attempt failures and print alert for top 2 suspected
ips

List & Dict Comprehensions

```
>>> l2 = [x for x in range(100)]
>>> l2
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99]
>>> d2 = {x:x**3 for x in range(100)}
>>> d2
{0: 0, 1: 1, 2: 8, 3: 27, 4: 64, 5: 125, 6: 216, 7: 343, 8: 512, 9: 729, 10: 1000, 11: 1331, 12: 1728, 13: 2197, 14: 2744, 15: 3375, 16: 4096, 17: 4913, 18: 5832, 19: 6859, 20: 8000, 21: 9261, 22: 10648, 23: 12167, 24: 13824, 25: 15625, 26: 17576, 27: 19683, 28: 21952, 29: 24389, 30: 27000, 31: 29791, 32: 32768, 33: 35937, 34: 39304, 35: 42875, 36: 46656, 37: 50653, 38: 54872, 39: 59319, 40: 64000, 41: 68921, 42: 74088, 43: 79507, 44: 85184, 45: 91125, 46: 97336, 47: 103823, 48: 110592, 49: 117649, 50: 125000, 51: 132651, 52: 140608, 53: 148877, 54: 157464, 55: 166375, 56: 175616, 57: 185193, 58: 195112, 59: 205379, 60: 216000, 61: 226981, 62: 238328, 63: 250047, 64: 262144, 65: 274625, 66: 287496, 67: 300763, 68: 314432, 69: 328509, 70: 343000, 71: 357911, 72: 373248, 73: 389017, 74: 405224, 75: 421875, 76: 438976, 77: 456533, 78: 474552, 79: 493039, 80: 512000, 81: 531441, 82: 551368, 83: 571787, 84: 592704, 85: 614125, 86: 636056, 87: 658503, 88: 681472, 89: 704969, 90: 729000, 91: 753571, 92: 778688, 93: 804357, 94: 830584, 95: 857375, 96: 884736, 97: 912673, 98: 941192, 99: 970299}
>>> d3 = {k:v for k,v in d2.items() if v%2 == 0}
>>> d3
{0: 0, 2: 8, 4: 64, 6: 216, 8: 512, 10: 1000, 12: 1728, 14: 2744, 16: 4096, 18: 5832, 20: 8000, 22: 10648, 24: 13824, 26: 17576, 28: 21952, 30: 27000, 32: 32768, 34: 39304, 36: 46656, 38: 54872, 40: 64000, 42: 74088, 44: 85184, 46: 97336, 48: 110592, 50: 125000, 52: 140608, 54: 157464, 56: 175616, 58: 195112, 60: 216000, 62: 238328, 64: 262144, 66: 287496, 68: 314432, 70: 343000, 72: 373248, 74: 405224, 76: 438976, 78: 474552, 80: 512000, 82: 551368, 84: 592704, 86: 636056, 88: 681472, 90: 729000, 92: 778688, 94: 830584, 96: 884736, 98: 941192}
>>> |
```

Tuples

- Just like a list → indexed, heterogeneous, sliced
- IMMUTABLE
- Python uses them internally

```
>>> t1 = (1,2,3,56,7,8,98)
>>> type(t1)
<class 'tuple'>
>>> t2 = 1,2,3
>>> t2
(1, 2, 3)
>>> type(t2)
<class 'tuple'>
>>> z,y,z = t2
>>> z
3
>>> y
2
>>> x
'VISHWAS K SINGH'
>>> x,y,z = t2
>>> x
1
>>> y
2
>>> z
3
>>> t3 = 1,2,7
>>> t3 > t2
True
>>> t4 = 1,1,1,1,2,3,4,5,6,1,1,1,1
>>> t4.count(1)
8
>>> t4.index(4)
6
>>> |
```

Set

```
>>> s1 = {1,2,3,45,6,7,8}
>>> type(s1)
<class 'set'>
>>> dir(set)
['__and__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getstate__', '__gt__', '__hash__', '__iand__', '__init__', '__init_subclass__', '__ior__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__rand__', '__reduce__', '__reduce_ex__', '__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update', 'discard', 'intersection', 'intersection_update', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference', 'symmetric_difference_update', 'union', 'update']
>>> s1.add(10)
>>> s1
{1, 2, 3, 6, 7, 8, 10, 45}
>>> s1.add(3)
>>> s1
{1, 2, 3, 6, 7, 8, 10, 45}
>>> l1 = [45,67,45,89,90,67,90]
>>> l2 = list(set(l1))
>>> l2
[89, 90, 67, 45]
>>> s2 = {8,9}
>>> s2.issubset(s1)
Traceback (most recent call last):
  File "<python-input-262>", line 1, in <module>
    s2.issubset(s1)
    ^^^^^^^^^^^^^
AttributeError: 'set' object has no attribute 'issubset'. Did you mean: 'issubset'?
>>> s2.issubset(s1)
False
>>> s1.issubset(s2)
False
>>> s1
{1, 2, 3, 6, 7, 8, 10, 45}
>>> s2
{8, 9}
>>> s1.intersection(s2)
{8}
```

→ Distinct, Homogeneous, just like mathematical sets

Functions

```
7  # Function Definition
8  def add(a,b):
9      ... return a+b
10
11 print(add(10,20)) # Function Call
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

POSTGRESQL QUERY RESULTS

```
c:\Users\VishwasKSingh\Workspace\ey-coh8-workspace>python 11_demo_func.py
30
```

Scope of variables

```
1  # Function Definition
2  def add(a,b):
3      print(f"locally Values of a,b in add: {a},{b}")
4      return a+b
5
6  def sub(a,b):
7      print(f"locally Values of a,b in sub: {a},{b}")
8      return a-b
9
10 a = 89
11 b = 100
12 print(f"Globally Values of a,b: {a},{b}")
13 print(add(10,20)) # Function Call
14 print(sub(50,20)) # Function Call
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRESQL QUERY RESULTS

```
c:\Users\VishwasKSingh\Workspace\ey-coh8-workspace>python 11_demo_func.py
Globally Values of a,b: 89,100
locally Values of a,b in add: 10,20
30
locally Values of a,b in sub: 50,20
30
```

Local and Global Scope

```
11_demo_func.py > sub
1  # Function Definition
2  m = 100
3  print(f"Globally Value of m: {m}")
4  def add(a,b):
5      m = 25
6      print(f"locally Values of a,b in add: {a},{b}")
7      print(f"locally Value of m in add: {m}")
8      return a+b
9
10 def sub(a,b):
11     print(f"locally Values of a,b in sub: {a},{b}")
12     print(f"locally Value of m in sub: {m}")
13     return a-b
14
15 a = 89
16 b = 100
17 print(f"Globally Values of a,b: {a},{b}")
18 print(add(10,20)) # Function Call
19 print(sub(50,20)) # Function Call
20 print(f"Globally Value of m: {m}")
21 # m = 700
```

We can modify the global value

```
11_demo_func.py > ...
1  # Function Definition
2  m = 100
3  print(f"Globally Value of m: {m}")
4  def add(a,b):
5      global m
6      m = 25
7      print(f"locally Values of a,b in add: {a},{b}")
8      print(f"locally Value of m in add: {m}")
9      return a+b
10
11  def sub(a,b):
12      print(f"locally Values of a,b in sub: {a},{b}")
13      print(f"locally Value of m in sub: {m}")
14      return a-b
15
16  a = 89
17  b = 100
18  print(f"Globally Values of a,b: {a},{b}")
19  print(add(10,20)) # Function Call
20  print(sub(50,20)) # Function Call
21  print(f"Globally Value of m: {m}")
```